

PYTHON I PROGRAMOWANIE WIELOWĄTKOWE

```
1  import threading
2
3  def uf():
4      print("-----")
5      print(threading.current_thread().name)
6      print(threading.get_ident())
7      print(r'\_____/' )
8      return
9
10 threads = []
11 for i in range(5):
12     t = threading.Thread(target=uf)
13     threads.append(t)
14     t.start()
```

PYTHON I PROGRAMOWANIE WIELOWĄTKOWE

```
PS C:\tmp\PYTHON\THREAD> .\prog1.py
-----
Thread-1
-----
16300
Thread-2
-----
-----
16672
\_____ /
-----
\_____ /
Thread-3
Thread-4
11280
3792
\_____ /
\_____ /
Thread-5
15628
\_____ /
PS C:\tmp\PYTHON\THREAD> ■
```

PYTHON I PROGRAMOWANIE WIELOWĄTKOWE

```
1  import threading
2
3  def uf(liczba1, liczba2):
4      print("Liczby: ", liczba1, liczba2)
5      print("Wynik = ", int(liczba1) + int(liczba2))
6      return
7
8  t = threading.Thread(target=uf, args=(2,3))
9  t.start()
```

PYTHON I PROGRAMOWANIE WIELOWĄTKOWE

```
1  import threading
2
3  def uf(liczba1, liczba2):
4      print("Liczby: ", liczba1, liczba2)
5      print("Wynik = ", liczba1 + liczba2)
6      return
7
8  threads = []
9  for i in range(5):
10     t = threading.Thread(target=uf, args=(2,3))
11     threads.append(t)
12     t.start()
```

PYTHON I PROGRAMOWANIE WIELOWĄTKOWE

```
PS C:\tmp\PYTHON\THREAD> . \prog3.py
Liczby: 2 3
Wynik = 5
Liczby: 2 3
Liczby: 2 3
Wynik = 5
Wynik = 5
Liczby: 2 3
Liczby: 2 3
Wynik = 5
Wynik = 5
```

Interpreter Pythona rozwiązuje problem działania wielu wątków w ten sposób, że tylko jeden wątek może znajdować się w stanie wykonania w dowolnym momencie. Jest to realizowane za pomocą pojedynczej blokady: **Global Interpreter Lock (GIL)**, która jest blokadą, pozwalającym tylko jednemu wątkowi na kontrolowanie interpretera Pythona w danym momencie.

PYTHON I PROGRAMOWANIE WIELOWĄTKOWE

```
1  import time
2  from threading import Thread
3
4  COUNT = 50000000
5
6  def countdown(n):
7      while n>0:
8          n -= 1
9
10 start = time.time()
11 countdown(COUNT)
12 end = time.time()
13
14 print('Time taken in seconds -', end - start)
```

Program jednowątkowy.
Na (starym) laptopie wykonuje się
około 5 sek.

PYTHON I PROGRAMOWANIE WIELOWĄTKOWE

```
1  import time
2  from threading import Thread
3
4  COUNT = 50000000
5
6  def countdown(n):
7      while n>0:
8          n -= 1
9
10 t1 = Thread(target=countdown, args=(COUNT//2,))
11 t2 = Thread(target=countdown, args=(COUNT//2,))
12
13 start = time.time()
14 t1.start()
15 t2.start()
16 t1.join()
17 t2.join()
18 end = time.time()
19
20 print('Time taken in seconds -', end - start)
```

A teraz podzielmy obliczenia
pomiędzy dwa wątki.
Czas wykonania jest podobny
(nieco poniżej) 5 sek.

PYTHON I PROGRAMOWANIE WIELOWĄTKOWE

```
1 import sys
2 print(sys.getswitchinterval())
```

Można odczytać „interwał czasowy przełączania wątku” interpretera: 0,005

Można też ustawić (dokumentacja):

<https://docs.python.org/3/library/sys.html>

`sys.setswitchinterval(interval)`

Set the interpreter's thread switch interval (in seconds). This floating-point value determines the ideal duration of the “timeslices” allocated to concurrently running Python threads. Please note that the actual value can be higher, especially if long-running internal functions or methods are used. Also, which thread becomes scheduled at the end of the interval is the operating system's decision. The interpreter doesn't have its own scheduler.

New in version 3.2.

PYTHON I PROGRAMOWANIE WIELOWĄTKOWE

Ciekawą graficzną wizualizację i dyskusję w wykorzystaniu wątków można znaleźć tutaj:

<http://dabeaz.blogspot.com/2010/01/python-gil-visualized.html>

The Python GIL Visualized

In preparation for my upcoming PyCON'2010 talk on "Understanding the Python GIL", I've been working on a variety of new material--including some graphical visualization of the GIL behavior described in my earlier [talk](#). I'm still experimenting, but check it out.

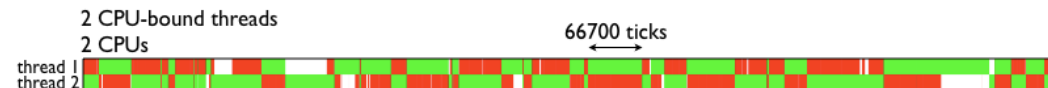
In these graphs, Python interpreter ticks are shown along the X-axis. The two bars indicate two different threads that are executing. White regions indicate times at which a thread is completely idle. Green regions indicate when a thread holds the GIL and is running. Red regions indicate when a thread has been scheduled by the operating system only to awake and find that the GIL is not available (e.g., the infamous "GIL Battle"). For those who don't want to read, here is the legend again in pictures:

□ Idle ■ Running ■ Failed GIL Acquire

Okay, now let's look at some threads. First, here is the behavior of running two CPU-bound threads on a single CPU system. As you will observe, the threads nicely alternate with each other after long periods of computation.



Now, let's go fire up the code on your fancy new dual-core laptop. Yow! Look at all of that GIL contention. Again, all of those red regions indicate times where the operating system has scheduled a Python thread on one of the cores, but it can't run because the thread on the other core is holding it.



PYTHON I PROGRAMOWANIE WIELOWĄTKOWE

```
1  import time
2  from multiprocessing import Pool
3
4  COUNT = 50000000
5
6  def countdown(n):
7      while n>0:
8          n -= 1
9
10 if __name__ == '__main__':
11     pool = Pool(processes=2)
12     start = time.time()
13     r1 = pool.apply_async(countdown, [COUNT//2])
14     r2 = pool.apply_async(countdown, [COUNT//2])
15     pool.close()
16     pool.join()
17     end = time.time()
18     print('Time taken in seconds -', end - start)
```

Najpopularniejszym sposobem ominięcia ograniczeń GIL jest użycie podejścia wieloprocusowego. Czas wykonania(około) 3 sek.

Appendix o logowaniu (logging)

```
1 import logging
2
3 logging.debug('This is a debug message')
4 logging.info('This is an info message')
5 logging.warning('This is a warning message')
6 logging.error('This is an error message')
7 logging.critical('This is a critical message')
```

```
WARNING:root:This is a warning message
ERROR:root:This is an error message
CRITICAL:root:This is a critical message
```

domyślny poziom istotności
WARNING i wyżej

Do monitorowania tego, co dzieje się w programie, można użyć moduł logging, korzystając z rangi istotności zdarzeń:

- DEBUG
- INFO
- WARNING
- ERROR
- CRITICAL

Appendix o logowaniu (logging)

Ustawianie poziomów ostrzeżeń za pomocą funkcji `basicConfig()`, z następującymi argumentami:

- **level**: poziom istotności
- **filename**: nazwa pliku
- **filemode**: jeśli podana nazwa pliku, to tu tryb jego otwarcia, domyślnie 'a' czyli append (dopisz)
- **format**: formatowanie wiadomości rejestrowanej

```
1 import logging
2 logging.basicConfig(filename='app.log', filemode='w', format='%(name)s - %(levelname)s - %(message)s')
3 logging.warning('This will get logged to a file')
```

w pliku `app.log` znajdziemy:

```
≡ app.log
1 root - WARNING - This will get logged to a file
```

PYTHON I PROGRAMOWANIE WIELOWĄTKOWE

Argumenty Thread: <https://docs.python.org/3/library/threading.html#threading.Thread>

docs.python.org/3/library/threading.html#threading.Thread

```
class threading.Thread(group=None, target=None, name=None, args=(), kwargs={}, *, daemon=None)
```

This constructor should always be called with keyword arguments. Arguments are:

group should be `None`; reserved for future extension when a `ThreadGroup` class is implemented.

target is the callable object to be invoked by the `run()` method. Defaults to `None`, meaning nothing is called.

name is the thread name. By default, a unique name is constructed of the form "Thread-*N*" where *N* is a small decimal number, or "Thread-*N* (*target*)" where "*target*" is `target.__name__` if the *target* argument is specified.

args is the argument tuple for the target invocation. Defaults to `()`.

kwargs is a dictionary of keyword arguments for the target invocation. Defaults to `{}`.

If not `None`, *daemon* explicitly sets whether the thread is daemonic. If `None` (the default), the daemonic property is inherited from the current thread.

If the subclass overrides the constructor, it must make sure to invoke the base class constructor (`Thread.__init__()`) before doing anything else to the thread.

Changed in version 3.10: Use the *target* name if *name* argument is omitted.

Changed in version 3.3: Added the *daemon* argument.

PYTHON I PROGRAMOWANIE WIELOWĄTKOWE

```
1  import time
2  import logging
3  import threading
4
5  def thread_function(name):
6      logging.info("Thread %s: starting", name)
7      time.sleep(2)
8      logging.info("Thread %s: finishing", name)
9
10 if __name__ == "__main__":
11     format = "%(asctime)s: %(message)s"
12     logging.basicConfig(format=format, level=logging.INFO, datefmt="%H:%M:%S")
13
14     logging.info("Main    : before creating thread")
15     x = threading.Thread(target=thread_function, args=(1,))
16     logging.info("Main    : before running thread")
17     x.start()
18     logging.info("Main    : wait for the thread to finish")
19     # x.join()
20     logging.info("Main    : all done")
```

```
19:33:36: Main      : before creating thread
19:33:36: Main      : before running thread
19:33:36: Thread 1: starting
19:33:36: Main      : wait for the thread to finish
19:33:36: Main      : all done
19:33:38: Thread 1: finishing
```

Thread 1 skończył po Main (zwróćmy uwagę na zakomentowane x.join())

PYTHON I PROGRAMOWANIE WIELOWĄTKOWE

Po uruchomieniu programu:

- przerwa (około 2 sekund) po wydrukowaniu przez `__main__` informacji o zakończeniu procesu, a przed zakończeniem wątku
- Python czeka na zakończenie wątku niebędącego demonem
- Po zakończeniu programu częścią procesu zamykania jest wyczyszczenie procedury wątkowania:
 - **`threading._shutdown()`** przebiega przez wszystkie działające wątki i wywołuje dla każdego wątku, który *nie ma ustawionej flagi `daemon`*, funkcję ich **`.join()`**
 - to właśnie powoduje czekanie głównego wątku na ich zakończenie

PYTHON I PROGRAMOWANIE WIELOWĄTKOWE

- powtórzmy z wątkiem demona: nie zobaczymy końca, demon zostanie zabity

```
x = threading.Thread(target=thread_function, args=(1,), daemon=True)
```

```
19:54:44: Main      : before creating thread
19:54:44: Main      : before running thread
19:54:44: Thread 1: starting
19:54:44: Main      : wait for the thread to finish
19:54:44: Main      : all done
```

- odkomentowanie `x.join()` spowoduje, że główny wątek będzie czekał z zakończeniem, aż Thread 1 skończy.

```
20:04:54: Main      : before creating thread
20:04:54: Main      : before running thread
20:04:54: Thread 1: starting
20:04:54: Main      : wait for the thread to finish
20:04:56: Thread 1: finishing
20:04:56: Main      : all done
```

PYTHON I PROGRAMOWANIE WIELOWĄTKOWE

```
1  import time
2  import logging
3  import threading
4
5  def thread_function(name):
6      logging.info("Thread %s: starting", name)
7      time.sleep(2)
8      logging.info("Thread %s: finishing", name)
9
10 if __name__ == "__main__":
11     format = "%(asctime)s: %(message)s"
12     logging.basicConfig(format=format, level=logging.INFO, datefmt="%H:%M:%S")
13
14     threads = list()
15     for index in range(3):
16         logging.info("Main      : create and start thread %d.", index)
17         x = threading.Thread(target=thread_function, args=(index,))
18         threads.append(x)
19         x.start()
20
21     for index, thread in enumerate(threads):
22         logging.info("Main      : before joining thread %d.", index)
23         thread.join()
24         logging.info("Main      : thread %d done", index)
```

Kod z trzema wątkami.
Kolejność **kończenia**
wątków jest losowa.

Istnieje łatwiejszy sposób na
rozpoczęcie grupy wątków:
ThreadPoolExecutor i jest częścią
standardowej biblioteki
w **concurrent.futures**

PYTHON I PROGRAMOWANIE WIELOWĄTKOWE

- powtórzmy z wątkiem demona: nie zobaczymy końca, demon zostanie zabity

```
x = threading.Thread(target=thread_function, args=(1,), daemon=True)
```

```
19:54:44: Main      : before creating thread
19:54:44: Main      : before running thread
19:54:44: Thread 1: starting
19:54:44: Main      : wait for the thread to finish
19:54:44: Main      : all done
```

- odkomentowanie `x.join()` spowoduje, że główny wątek będzie czekał z zakończeniem, aż Thread 1 skończy.

```
20:04:54: Main      : before creating thread
20:04:54: Main      : before running thread
20:04:54: Thread 1: starting
20:04:54: Main      : wait for the thread to finish
20:04:56: Thread 1: finishing
20:04:56: Main      : all done
```

Appendix o konstrukcji with

Aby wykorzystać sprawnie PoolExecutos, warto przypomnieć konstrukcję językową „with”, na kilku przykładach:

1) bez użycia with

```
file = open('file_path', 'w')
file.write('hello world !')
file.close()
```

2) bez with ale z wjątkami

```
file = open('file_path', 'w')
try:
    file.write('hello world')
finally:
    file.close()
```

3) użycie with

```
with open('file_path', 'w') as file:
    file.write('hello world !')
```

PYTHON I PROGRAMOWANIE WIELOWĄTKOWE

```
1  import logging
2  #import threading
3  import concurrent.futures
4  import time
5
6  def thread_function(name):
7      logging.info("Thread %s: starting", name)
8      time.sleep(2)
9      logging.info("Thread %s: finishing", name)
10
11  if __name__ == "__main__":
12      format = "%(asctime)s: %(message)s"
13      logging.basicConfig(format=format, level=logging.INFO, datefmt="%H:%M:%S")
14
15  with concurrent.futures.ThreadPoolExecutor(max_workers=3) as executor:
16      executor.map(thread_function, range(3))
```

ThreadPoolExecutor

```
18:48:17: Thread 0: starting
18:48:17: Thread 1: starting
18:48:17: Thread 2: starting
18:48:19: Thread 2: finishing
18:48:19: Thread 0: finishing
18:48:19: Thread 1: finishing
```

- trzy wątki kończą w losowej kolejności

PROBLEMY PROGRAMOWANIA WIELOWĄTKOWEGO – DATA RACE

```
1  import logging
2  import concurrent.futures
3  import time
4
5  class FakeDatabase:
6      def __init__(self):
7          self.value = 0
8
9      def update(self, name):
10         logging.info("Thread %s: starting update", name)
11         local_copy = self.value
12         local_copy += 1
13         time.sleep(0.1)
14         self.value = local_copy
15         logging.info("Thread %s: finishing update", name)
```

Do zademonstrowania
głównego problemu
programowania
wielowątkowego użyjemy
obiektu "bazy danych" i jednej
składowej zmiennej .value

PROBLEMY PROGRAMOWANIA WIELOWĄTKOWEGO – DATA RACE

```
17 if __name__ == "__main__":
18     format = "%(asctime)s: %(message)s"
19     logging.basicConfig(format=format, level=logging.INFO,
20                         datefmt="%H:%M:%S")
21
22     database = FakeDatabase()
23     logging.info("Testing update. Starting value is %d.", database.value)
24     with concurrent.futures.ThreadPoolExecutor(max_workers=2) as executor:
25         for index in range(2):
26             executor.submit(database.update, index)
27     logging.info("Testing update. Ending value is %d.", database.value)
```

Operacja wykonywana na dwóch wątkach.

`.submit(function, *args, **kwargs)`

Wynik końcowy (wartość 1) nie jest zgodny z oczekiwaniem (wartość 2).

```
19:46:35: Testing update. Starting value is 0.
19:46:35: Thread 0: starting update
19:46:35: Thread 1: starting update
19:46:35: Thread 1: finishing update
19:46:35: Thread 0: finishing update
19:46:35: Testing update. Ending value is 1.
```

PROBLEMY PROGRAMOWANIA WIELOWĄTKOWEGO – DATA RACE

Rozwiązanie stanu wyścigu (data race)

- tylko jeden wątek może w sekcji odczytywać, modyfikować i zapisywać
- najpopularniejszym sposobem na to jest blokada Lock
- funkcje zdobywania i zwalniania blokady to `.acquire ()` i `.release ()`
- wątek wywoła funkcję `my_lock.acquire()`, aby uzyskać blokadę. Jeśli blokada jest już wstrzymana, wątek wywołujący będzie czekał, aż zostanie zwolniony. Jeśli jeden wątek otrzyma blokadę, ale nigdy jej nie zwróci, program się zaklinuje (**deadlock**).
- Lock działa też jako menedżer kontekstu, więc można go używać w instrukcji **with** i jest automatycznie zwalniany, gdy blok **with** z dowolnego powodu zostanie zakończony.

PROBLEMY PROGRAMOWANIA WIELOWĄTKOWEGO – DATA RACE

```
1 import logging
2 import threading
3 import concurrent.futures
4 import time
5
6 class FakeDatabase:
7     def __init__(self):
8         self.value = 0
9         self._lock = threading.Lock()
10
11     def locked_update(self, name):
12         logging.info("Thread %s: starting update", name)
13         logging.debug("Thread %s about to lock", name)
14         with self._lock:
15             logging.debug("Thread %s has lock", name)
16             local_copy = self.value
17             local_copy += 1
18             time.sleep(0.1)
19             self.value = local_copy
20             logging.debug("Thread %s about to release lock", name)
21         logging.debug("Thread %s after release", name)
22         logging.info("Thread %s: finishing update", name)
```

```
21:54:59: Testing update. Starting value is 0.
21:54:59: Thread 0: starting update
21:54:59: Thread 1: starting update
21:54:59: Thread 0: finishing update
21:54:59: Thread 1: finishing update
21:54:59: Testing update. Ending value is 2.
```

Wynik końcowy (wartość 2) jest teraz zgodny z oczekiwaniem.

PROBLEMY PROGRAMOWANIA WIELOWĄTKOWEGO – DEADLOCK

```
1  import threading
2
3  l = threading.Lock()
4  print("before first acquire")
5  l.acquire()
6  print("before second acquire")
7  l.acquire()  # l.acquire(True,1)  albo l.acquire(False)
8
9  print("acquired lock twice")
```

Argumenty **acquire** dla Lock: <https://docs.python.org/3/library/threading.html#threading.Lock>

Jeśli Lock został już uzyskany, drugie wywołanie `.acquire()` będzie czekało, aż wątek, w którym znajduje się blokada, wywoła `.release()`

Można skorzystać z opcji gdy po określonym czasie próba zostanie zaniechana.

PROGRAMOWANIE WIELOWĄTKOWE – RLock

W sytuacji, gdy oddzielny kod z tego samego wątku musi „ponownie uzyskać” blokadę, zamiast prostego `threading.Lock()` należy użyć `threading.RLock`

```
1  import threading
2
3  l = threading.RLock()
4  print("before first acquire")
5  l.acquire()
6  print("before second acquire")
7  l.acquire(0)
8  print("acquired lock twice")
```

RLock: <https://docs.python.org/3/library/threading.html#rlock-objects>

PROGRAMOWANIE WIELOWĄTKOWE – CONCURRENT

concurrent.futures

- concurrent.futures, dodany w Python 3.2
- interfejs wysokiego poziomu do uruchamiania zadań asynchronicznych
- interfejs do jednoczesnego wykonywania zadań przy użyciu pul **wątków** lub **procesów**

Executor (Abstract Base Class)

└─ **ThreadPoolExecutor**

|

| | do zarządzania zadaniami związanymi z I/O, za pomocą wątków

|

└─ **ProcessPoolExecutor**

|

| | do zarządzania zadaniami związanymi z obliczeniami na CPU, za pomocą procesów

PROGRAMOWANIE WIELOWĄTKOWE – SUBMIT, MAP

ThreadPoolExecutor, jak i ProcessPoolExecutor mają ten sam interfejs API, oraz dwie ważne metody:

- `submit(fn, args, *kwargs)`
 - wywołanie `fn(* args ** kwargs)` zwraca obiekt `Future` reprezentujący wykonanie wywoływanego elementu

```
1  with concurrent.futures.ThreadPoolExecutor(max_workers=1) as executor:
2      future = executor.submit(pow, 323, 1235)
3      print(future.result())
```

- `map(func, *iterables, timeout=None, chunksize=1)`

```
6  import time
7  import concurrent.futures
8
9  e = concurrent.futures.ThreadPoolExecutor(4)
10 s = range(10)
11 ✓ for i in e.map(time.sleep, s):
12     print(i)
```

dla `ProcessPoolExecutor` ta metoda dzieli iteracje na kilka fragmentów, które przesyła do puli jako oddzielne zadania. (Przybliżony) rozmiar tych fragmentów można określić, ustawiając rozmiar **chunksize** na dodatnią liczbę całkowitą

PROGRAMOWANIE WIELOWĄTKOWE – SUBMIT

```
1  for task in get_tasks():
2      perform(task)
```

pula zadań do wykonania

```
1  import concurrent.futures
2
3  with concurrent.futures.Executor() as executor:
4      futures = {executor.submit(perform, task): task for task in get_tasks()}
5
6      for fut in concurrent.futures.as_completed(futures):
7          original_task = futures[fut]
8          print(f"The result of {original_task} is {fut.result()}")
```

moduł wykonawczy

- w prawdziwym kodzie należałoby zamienić Executor na ThreadPoolExecutor lub ProcessPoolExecutor
- metoda executor.**submit()** planuje asynchroniczne wykonanie zadania, tworzy obiekt Future, który reprezentuje zadanie do wykonania
- po zaplanowaniu wszystkich zadań wywoływana jest metoda concurrent.futures.**as_completed()**, która zwraca przyszłe wyniki w miarę ich wykonania - czyli po zakończeniu każdego zadania
- metoda fut.**result()** zwraca wartość perform(task) lub zgłasza wyjątek w przypadku niepowodzenia

as_completed: https://docs.python.org/3/library/concurrent.futures.html?highlight=as_completed#concurrent.futures.as_completed

PROGRAMOWANIE WIELOWĄTKOWE – MAP

```
1  for task in get_tasks():
2      perform(task)
```

pula zadań do wykonania

```
1  import concurrent.futures
2
3  with concurrent.futures.Executor() as executor:
4      for arg, res in zip(get_tasks(), executor.map(perform, get_tasks())):
5          print(f"The result of {arg} is {res}")
```

moduł wykonawczy

- inny sposób gromadzenia wyników w tej samej kolejności, w jakiej zostały zaplanowane to użycie metody **executor.map()**
- wyniki zwracany jest w tej samej kolejności, w jakiej zostało zaplanowane obliczenie
- jeśli jakikolwiek nieobsługiwany wyjątek wystąpi podczas operacji, zostanie on również natychmiast zgłoszony, a wykonanie nie będzie kontynuowane

PROGRAMOWANIE WIELOWĄTKOWE – SUBMIT, MAP

```
1  import time
2  import concurrent.futures
3
4  def worker(i):
5      time.sleep(i)
6      return i,time.time()
7
8  e = concurrent.futures.ThreadPoolExecutor(2)
9  arrIn = range(1,7)[::-1]
10 print(arrIn)
11
12 f = []
13 print('start submit',time.time())
14 for i in arrIn:
15     f.append(e.submit(worker,i))
16 print('submitted',time.time())
17 for r in concurrent.futures.as_completed(f):
18     print(r.result(),time.time())
19 print
20
21 f = []
22 print('start map',time.time())
23 f = e.map(worker,arrIn)
24 print('mapped',time.time())
25 for r in f:
26     print(r,time.time())
```

iteracja po liście obiektów Future z `concurrent.futures.as_completed()` nie gwarantuje kolejności, ponieważ taka jest natura `as_completed()`

`.map()` daje wyniki w kolejności, w jakiej są przesyłane

DEKORATOR DO POMIARU CZASU

```
1  import time
2  from functools import wraps
3
4
5  def timeit(method):
6      @wraps(method)
7      def wrapper(*args, **kwargs):
8          start_time = time.time()
9          result = method(*args, **kwargs)
10         end_time = time.time()
11         print(f"{method.__name__} => {(end_time-start_time)*1000} ms")
12
13         return result
14
15     return wrapper
16
17 @timeit
18 def func(n):
19     time.sleep(n)
20
21 func(3)
```

SEKWENCYJNE ŚCIAĞANIE PLIKÓW

```
def download_one(url):  
    req = urllib.request.urlopen(url)  
    fullpath = Path(url)  
    fname = fullpath.name  
    ext = fullpath.suffix  
  
    if not ext:  
        raise RuntimeError("URL does not contain an extension")  
  
    with open(fname, "wb") as handle:  
        while True:  
            chunk = req.read(1024)  
            if not chunk:  
                break  
            handle.write(chunk)  
  
    msg = f"Finished downloading {fname}"  
    return msg
```

```
@timeit  
def download_all(urls):  
    return [download_one(url) for url in urls]  
  
if __name__ == "__main__":  
    urls = (  
        "http://zfj.if.uj.edu.pl/tmp/Python1.pdf",  
        "http://zfj.if.uj.edu.pl/tmp/Python2.pdf",  
        "http://zfj.if.uj.edu.pl/tmp/Python3.pdf",  
        "http://zfj.if.uj.edu.pl/tmp/Python4.pdf",  
        "http://zfj.if.uj.edu.pl/tmp/Python5.pdf",  
        "http://zfj.if.uj.edu.pl/tmp/Python6.pdf",  
        "http://zfj.if.uj.edu.pl/tmp/Python7.pdf",  
        "http://zfj.if.uj.edu.pl/tmp/Python8.pdf",  
    )  
  
    results = download_all(urls)  
    for result in results:  
        print(result)
```

czas: 3-4 sekundy

SEKWENCYJNE ŚCIAĞANIE PLIKÓW – WERSJA WIELOWĄTKOWA I MAP

```
from concurrent.futures import
ThreadPoolExecutor, as_completed

@timeit
def download_all(urls):

    with ThreadPoolExecutor(max_workers=8) as executor:
        return executor.map(download_one, urls, timeout=60)

if __name__ == "__main__":
    urls = (
        "http://zfj.if.uj.edu.pl/tmp/Python1.pdf",
        "http://zfj.if.uj.edu.pl/tmp/Python2.pdf",
        "http://zfj.if.uj.edu.pl/tmp/Python3.pdf",
        "http://zfj.if.uj.edu.pl/tmp/Python4.pdf",
        "http://zfj.if.uj.edu.pl/tmp/Python5.pdf",
        "http://zfj.if.uj.edu.pl/tmp/Python6.pdf",
        "http://zfj.if.uj.edu.pl/tmp/Python7.pdf",
        "http://zfj.if.uj.edu.pl/tmp/Python8.pdf",
    )

    results = download_all(urls)
    for result in results:
        print(result)
```

wersja z ThreadPoolExecutor

mniej wydajna

czas: < 1 sekundy

wersja z ProcessPoolExecutor

mniej wydajna

czas: 2,6 sekundy

Ile wątków użyć? Empiryczna zasada:
2 * multiprocessing.cpu_count() + 1

SEKWENCYJNE ŚCIAĞANIE PLIKÓW – WERSJA WIELOWĄTKOWA I SUBMIT

```
from concurrent.futures import  
ThreadPoolExecutor, as_completed
```

```
@timeit  
def download_all(urls):  
  
    futures_list = []  
    results = []  
  
    with ThreadPoolExecutor(max_workers=13) as executor:  
        for url in urls:  
            futures = executor.submit(download_one, url)  
            futures_list.append(futures)  
  
        for future in futures_list:  
            try:  
                result = future.result(timeout=60)  
                results.append(result)  
            except Exception:  
                results.append(None)  
    return results
```

czas: < 1 sekundy

PROGRAM OBCIĄŻAJĄCY CPU – WERSJA SEKWENCYJNA

```
import hashlib
```

```
def hash_one(n):  
    """ CPU-obciazenie """  
  
    for i in range(1, n):  
        hashlib.pbkdf2_hmac("sha256", b"password", b"salt", i * 10000)  
  
    return "done"
```

```
@timeit  
def hash_all(n):  
  
    for i in range(n):  
        hsh = hash_one(n)  
  
    return "done"
```

```
if __name__ == "__main__":  
    hash_all(15)
```

czas: > 18 sekund

PROGRAM OBCIĄŻAJĄCY CPU – WERSJA RÓWNOLEGŁA (PROCESS)

```
from concurrent.futures import ProcessPoolExecutor
```

```
def hash_one(n):  
    """CPU-obciazenie"""  
  
    for i in range(1, n):  
        hashlib.pbkdf2_hmac("sha256", b"password", b"salt", i * 10000)  
  
    return "done"
```

```
@timeit  
def hash_all(n):  
  
    with ProcessPoolExecutor(max_workers=10) as executor:  
        for arg, res in zip(range(n), executor.map(hash_one, range(n), chunksize=1)):  
            pass  
  
    return "done"
```

```
if __name__ == "__main__":  
    hash_all(15)
```

czas: 4,6 sekundy

PROGRAM Z THREADS – ZAKLESZCZENIE

```
import time
from concurrent.futures import ThreadPoolExecutor

def wait_on_b():
    time.sleep(5)
    print(b.result()) # b will never complete because it is waiting on a.
    return 5

def wait_on_a():
    time.sleep(5)
    print(a.result()) # a will never complete because it is waiting on b.
    return 6

with ThreadPoolExecutor(max_workers=2) as executor:
    # here, the future from a depends on the future from b
    # and vice versa
    # so this is never going to be completed
    a = executor.submit(wait_on_b)
    b = executor.submit(wait_on_a)

    print("Result from wait_on_b", a.result())
    print("Result from wait_on_a", b.result())
```

czas: "nieskończony"

PROGRAM Z THREADS – ZAKLESZCZENIE (JEDEN WĄTEK!)

```
from concurrent.futures import ThreadPoolExecutor

def wait_on_future():
    f = executor.submit(pow, 5, 2)
    # This will never complete because there is only one worker thread and
    # it is executing this function.
    print(f.result())

with ThreadPoolExecutor(max_workers=1) as executor:
    future = executor.submit(wait_on_future)
    print(future.result())
```

czas: "nieskończony" dla 1 wątku
działa dla: max_workers=2

PROGRAM – LICZBY PIERWSZE (SEKWENCYJNIE)

```
import math
```

```
PRIMES = [num for num in range(19000, 20000)]
```

```
def is_prime(n):  
    if n < 2:  
        return False  
    if n == 2:  
        return True  
    if n % 2 == 0:  
        return False  
  
    sqrt_n = int(math.floor(math.sqrt(n)))  
    for i in range(3, sqrt_n + 1, 2):  
        if n % i == 0:  
            return False  
    return True
```

```
@timeit
```

```
def main():  
    for number in PRIMES:  
        print(f"{number} is prime: {is_prime(number)}")
```

```
if __name__ == "__main__":  
    main()
```

czas: 2 sekundy

PROGRAM – LICZBY PIERWSZE (WSPÓŁBIEŻNIE)

```
from concurrent.futures import ProcessPoolExecutor, ThreadPoolExecutor
```

```
PRIMES = [num for num in range(19000, 20000)]  
num_list = [num for num in range(19000, 20000)]
```

```
def is_prime(n):  
    if n < 2:  
        return False  
    if n == 2:  
        return True  
    if n % 2 == 0:  
        return False  
  
    sqrt_n = int(math.floor(math.sqrt(n)))  
    for i in range(3, sqrt_n + 1, 2):  
        if n % i == 0:  
            return False  
    return True
```

```
@timeit  
def main():  
    with ThreadPoolExecutor(max_workers=10) as executor:  
        for number, prime in zip(PRIMES, executor.map(is_prime, num_list)):  
            print(f"{number} is prime: {prime}")
```

```
if __name__ == "__main__":  
    main()
```

czas: dłuższy np. 6 sekund

Intuicyjnie może się wydawać, że zadanie sprawdzania liczb pierwszych powinno być operacją związaną z procesorem, ważne jest również ustalenie, czy samo zadanie jest wystarczająco obciążone obliczeniowo, aby uzasadnić tworzenie wielu wątków lub procesów.

PROGRAM – KOSZT PRZETWARZANIA (SERYJNIE)

```
import random
def do_something(count, out_list):
    for i in range(count):
        out_list.append(random.random())
```

zapiszmy w pliku losowanie.py

```
from losowanie import *
import time
```

```
if __name__ == "__main__":
    start_time = time.time()
    size = 10000000
    n_exec = 10
    for i in range(0, n_exec):
        out_list = list()
        do_something(size, out_list)
    print("List processing complete.")
    end_time = time.time()
    print("serial time=", end_time - start_time)
```

czas: 24 sekundy

PROGRAM – KOSZT PRZETWARZANIA (WIELOWĄTKOWO)

```
from losowanie import *
import time
import threading

if __name__ == "__main__":
    start_time = time.time()
    size = 100000000
    threads = 10
    jobs = []
    for i in range(0, threads):
        out_list = list()
        thread = threading.Thread(target=do_something(size, out_list))
        jobs.append(thread)
    for j in jobs:
        j.start()
    for j in jobs:
        j.join()
    print("List processing complete.")
    end_time = time.time()
    print("multithreading time=", end_time - start_time)
```

czas: 24 sekundy
(podobny do sekwencyjnego)

PROGRAM – KOSZT PRZETWARZANIA (WIELOPROCESOWO)

```
from losowanie import *
import time
import multiprocessing

if __name__ == "__main__":
    start_time = time.time()
    size = 10000000
    procs = 10
    jobs = []
    for i in range(0, procs):
        out_list = list()
        process = multiprocessing.Process \
            (target=do_something, args=(size, out_list))
        jobs.append(process)

    for j in jobs:
        j.start()
    for j in jobs:
        j.join()

    print("List processing complete.")
    end_time = time.time()
    print("multiprocesses time=", end_time - start_time)
```

czas: 24 sekundy
(podobny do sekwencyjnego)

PROGRAM – KOSZT PRZETWARZANIA – WNIOSKI

Zdarzają się sytuacje, w których wydajność (czas wykonania) programu może być niższa w przypadku kodu współbieżnego niż jego sekwencyjnego odpowiednika.

Może się to zdarzyć z wielu powodów:

- wątki były używane do wykonywania zadań związanych z procesorem CPU
- wieloprosesowość była używana do wykonywania zadań związanych z I/O
- zadania były zbyt trywialne, aby uzasadniać użycie wątków lub wielu procesów

Odtwarzanie / usypianie wielu wątków lub procesów wiąże się z dodatkowymi kosztami. Zwykle wątki są w kontekście tych kosztów znacznie szybsze niż procesy.

Użycie niewłaściwego typu współbieżności może w rzeczywistości spowolnić kod lub zysk będzie marginalny (kosztem komplikacji kodu).